

LAB 08 • PART II • SUPPORT / SERVICE

Inbound Responder

Turn Ghost into a support agent that answers from a knowledge base you give it — and escalates to a human the moment it hits the edge of what it knows, instead of guessing.

DIFFICULTY • MEDIUM

TIME • ~75 MIN

FUNCTION • SUPPORT / SERVICE

BUILDS ON • MEMORY (L1) + GUARDRAILS (L4)

OBJECTIVES

- Load a real FAQ / policy set into Ghost's memory as the **single source of truth** for a support channel.
- Have Ghost answer inbound questions in a Discord channel using *only* that knowledge base — short, accurate, no embroidery.
- Install the boundary rule: when an answer isn't in the base or confidence is low, say so plainly and **flag for a human**.
- Run the escalation test — ask something out of scope and confirm Ghost escalates rather than fabricates.
- Crystallize the behavior into a saved "responder" workflow you can reuse on any support channel.

Why this lab exists

Every other lab in this course points Ghost *outward* or *inward* — drafting your emails, running your research, managing your own work. This is the one lab where Ghost faces **other people**. Inbound support is the archetypal field deployment: a customer or teammate asks a question in a channel, and an agent has to answer it correctly, quickly, and safely, with no one checking its homework first.

That changes the posture completely. Outbound agents can be creative; a support agent must be *grounded*. The single most valuable instinct a production agent can have is knowing the boundary of its own knowledge. A support bot that refuses gracefully — "I don't have that in my knowledge base, I've flagged a human" — is deployable today. A support bot that confidently invents a refund policy is a liability that will eventually cost someone money or trust. This lab builds the reflex we care about most: **escalate, don't hallucinate**.

Prerequisites

- Part I complete — you have a Ghost live and responding in a Discord channel.
- **Lab 1 (Memory)** especially: you know how to load durable facts into Ghost's memory and confirm they stuck.
- **Lab 4 (Guardrails)** especially: you've written a hard rule for Ghost and watched it hold under pressure.
- A small, real knowledge base to hand it: 8–15 FAQ entries, a policy doc, or a product help page. It doesn't have to be big — it has to be *true*.

The build — step by step

Four beats. You **Prime** the knowledge base, **Prompt** for grounded answers, **Break & steer** with an out-of-scope question, then **Crystallize** the whole behavior into a saved workflow.

1

Prime — load the knowledge base as the single source of truth

Paste your FAQ / policy content straight into the channel and have Ghost commit it to memory. Be explicit that this is *the* source — not background reading, but the only ground it's allowed to stand on.

```
# PRIME THE KNOWLEDGE BASE INTO MEMORY
```

```
Ghost, this is the official support knowledge base for our product. Save it to your memory as the single source of truth for this channel. Do not blend it with anything you already know from the web.
```

```
--- KNOWLEDGE BASE ---
```

```
Q: How long is the free trial? A: 14 days, no card required.
```

```
Q: What's the refund window? A: 30 days from purchase, full refund, no questions asked.
```

```
Q: Which regions do we ship to? A: US, Canada, UK, and the EU.
```

```
Q: How do I reset my password? A: Settings > Security > Reset password; link expires in 1 hour.
```

```
...(paste the rest of your entries here)...
```

```
--- END ---
```

```
When you've stored it, list back the topics you now consider authoritative so I can confirm nothing is missing.
```

Read the topic list it returns. If an entry didn't land, re-paste it now — a gap in the base becomes a hallucination later.

Prompt — answer inbound questions from the base only

Now set the answering behavior. The rule that matters: replies come *only* from the stored base, stay short, and never pad an answer with plausible-sounding extras.

```
# SET THE GROUNDED ANSWERING BEHAVIOR
```

```
From now on, when someone asks a question in this channel, answer it using ONLY the knowledge base you just stored. Rules:
```

- Keep replies to 1-3 sentences. Accurate over friendly.
- Quote the policy as written. Do not soften, round, or extend it.
- Never add details that aren't in the base, even if they sound reasonable.

```
Let's test. A customer asks: "Can I get a refund three weeks after I bought it?"
```

Three weeks is inside the 30-day window, so Ghost should answer "yes" and cite the 30-day policy. Try two or three more in-scope questions and confirm every answer traces back to a real entry — no invented specifics.

3

Break & steer — set the boundary, then test it

This is the muscle of the lab. First, state the boundary rule explicitly. Then attack it with a question the base can't answer.

```
# INSTALL THE ESCALATE-DON'T-HALLUCINATE RULE
```

```
Critical rule for this channel: if a question's answer is NOT in the knowledge base,
or you're not confident, do NOT guess and do NOT invent a policy. Instead reply
plainly:
```

```
"I don't have that in my knowledge base — I've flagged this for a human to answer,"
and post a short @human escalation note with the exact question. Refusing correctly is
a success, not a failure.
```

```
# THE ESCALATION TEST — AN OUT-OF-SCOPE QUESTION
```

```
Customer question in the channel: "Do you offer a student discount, and how much is
it?"
```

There is no discount entry in the base. Ghost must **escalate** — say it doesn't know and flag a human — not manufacture a percentage. If it invents anything, harden the rule and re-test: "You just invented a discount that isn't in the base. That's the one thing you must never do. Restate the boundary rule and try that question again." Repeat until a clean refusal is reliable across a few different out-of-scope questions.

4

Crystallize — save the responder workflow

Fold grounding + escalation into one reusable behavior saved to Ghost, so any future support channel can inherit it.

```
# SAVE THE REUSABLE RESPONDER WORKFLOW
```

```
Save this as a workflow called "Inbound Responder." It should capture:
```

1. Answer inbound questions from a designated knowledge base only, in 1-3 sentences.
2. Never add details not in the base.
3. If the answer isn't in the base or confidence is low, refuse plainly and escalate

```
to
```

```
    a human with the exact question.
```

```
Confirm it's saved so I can point it at a new channel and just load a fresh base.
```

THE ESCALATION TEST — WHAT FAILURE LOOKS LIKE

The whole lab hinges on Step 3. When you ask the out-of-scope question, watch for these failure modes and steer until they're gone:

- **Confident fabrication** — Ghost states a discount, a number, or a policy that appears nowhere in the base. This is the cardinal sin. Harden the rule and re-test.
- **Web leakage** — Ghost answers from general web knowledge instead of your base ("most companies offer 10%"). The base is the *only* ground; remind it explicitly.
- **Silent guessing dressed as certainty** — a plausible answer with no citation and no hedge. If it can't point to a KB entry, it must escalate.
- **Over-escalation** — flagging a human for questions the base clearly does answer. A refusal is only correct when the base is genuinely silent; recalibrate so it answers what it knows.

A pass looks like: for an in-scope question, a short cited answer; for an out-of-scope question, a plain "I don't have that — flagged for a human" with the question handed off. Nothing invented, either way.

DELIVERABLE

- ☐ A knowledge base loaded into Ghost's memory as the single source of truth, with its stored topics confirmed.
- ☐ A working responder live in a Discord channel that answers in-scope questions accurately and briefly, straight from the base.
- ☐ A **logged example** of Ghost correctly escalating an out-of-scope question — the question, its plain refusal, and the human hand-off, captured for the record.
- ☐ A saved "Inbound Responder" workflow carrying the grounding + escalation rules, ready to point at a new channel.

Stretch goals

- **Cite the source.** Have Ghost name which KB entry each answer came from ("per the 30-day refund policy"), so answers are auditable at a glance.
- **Confidence tag.** Add a *confidence: high / low* marker to every reply; a low tag becomes an automatic candidate for escalation.
- **Escalation log for KB improvement.** Have Ghost keep a running list of every escalated question in memory — each one is a gap in the base and a candidate new FAQ entry to add later.

FACILITATOR NOTES

The learning goal is the *reflex*, not the FAQ. Students often over-invest in a big knowledge base; steer them to a small, true one and spend the time on Step 3. The moment worth engineering is watching Ghost refuse a plausible question — that's when the "escalate, don't hallucinate" instinct becomes real to them.

Common sticking point: Ghost answers out-of-scope questions from general web knowledge because it feels helpful. Reframe helpfulness for the group — in support, the most helpful thing an unsure agent can do is hand off cleanly. Reward the refusal out loud so students stop reading it as failure.

Keep it inside the envelope: everything happens in Discord (or from a pasted form) and in Ghost's own memory and cloud workspace — no local files, no external ticketing integration, and don't promise it will answer on a schedule. If a student wants automatic routing to a real human queue, note it as a future integration and keep this lab focused on the boundary behavior. Time check: aim for ~20 min priming, ~40 min on prompt + the escalation loop, ~15 min to crystallize and log the deliverable.